# CodeAlpha Internship — Project Execution Evidence

## Distributed Build Execution Using Jenkins Remoting

| | |
|---|---|
| **Candidate / Track:** CodeAlpha DevOps Intern | **Environment:** Windows Host (Adoptium JDK 21) & Docker Engine |
| **Project Domain:** CI/CD & Distributed Systems | **Controller Version:** Jenkins LTS (JDK 21) via JCasC |
| **Remoting Protocols:** JNLP4-connect (TCP 50000) & WebSocket | **Execution Workspaces:** Isolated agent-specific directories |

## Project Purpose & Environmental Overview

This project establishes a resilient, production-ready **Distributed Build Architecture** using Jenkins Remoting. To eliminate security vulnerabilities and performance bottlenecks inherent to monolithic CI/CD setups, the Jenkins Controller is configured with zero executors (numExecutors: 0). All build, compilation, and test execution workloads are delegated strictly to remote inbound worker nodes authenticated via genuine HMAC cryptographic tokens.

**Important Environment Topology Notice:** The Linux inbound agent (running inside an isolated Docker container with Temurin JDK 21) and the Windows inbound agent (running as a native Windows host process with Temurin JDK 21) represent separate **operating-system runtime environments** hosted on the same workstation, not physically distinct hardware machines or disparate CPU architectures. Both environments successfully connect to the central Controller, enforce separated workspace boundaries, and follow least-privilege principles.

## Evidence #1: Jenkins Controller Dashboard & Zero Built-In Executors

The Jenkins Controller dashboard displays the codealpha-distributed-job pipeline in a healthy state (blue indicator). The *Build Executor Status* panel confirms that the Built-In Node has exactly zero executors (numExecutors: 0), verifying that the controller is restricted purely to orchestration and cannot execute user-submitted build scripts.

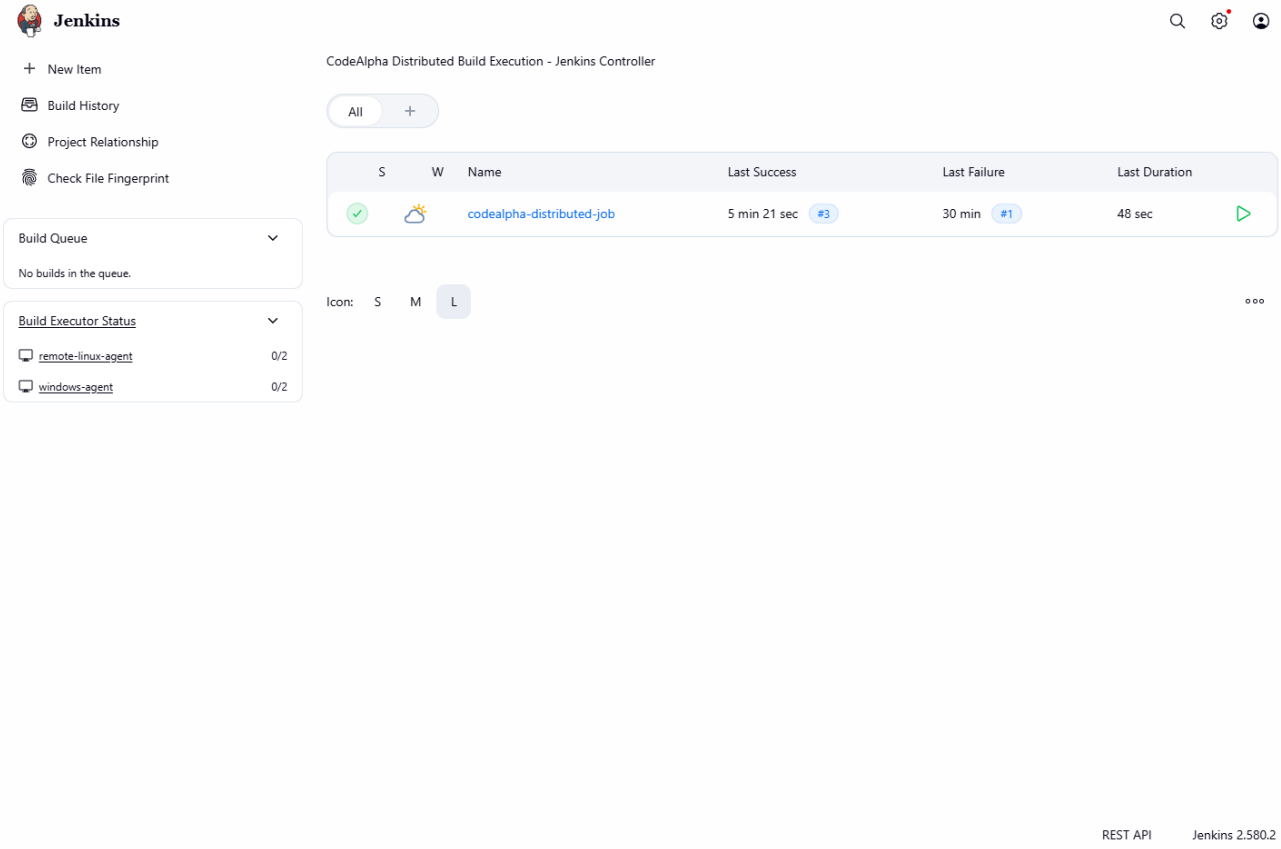



*Figure 1: Actual screenshot captured from live running Jenkins environment (File: 01-jenkins-controller-dashboard.png).*

## Evidence #2: Jenkins Node Management — Distributed Nodes Overview

The Node Management interface (/computer/) confirms that both distributed agents—remote-linux-agent (2 executors) and windows-agent (2 executors)—are registered, in sync, and actively connected. The total cluster execution capacity is 4 distributed executors with 0 on the controller.

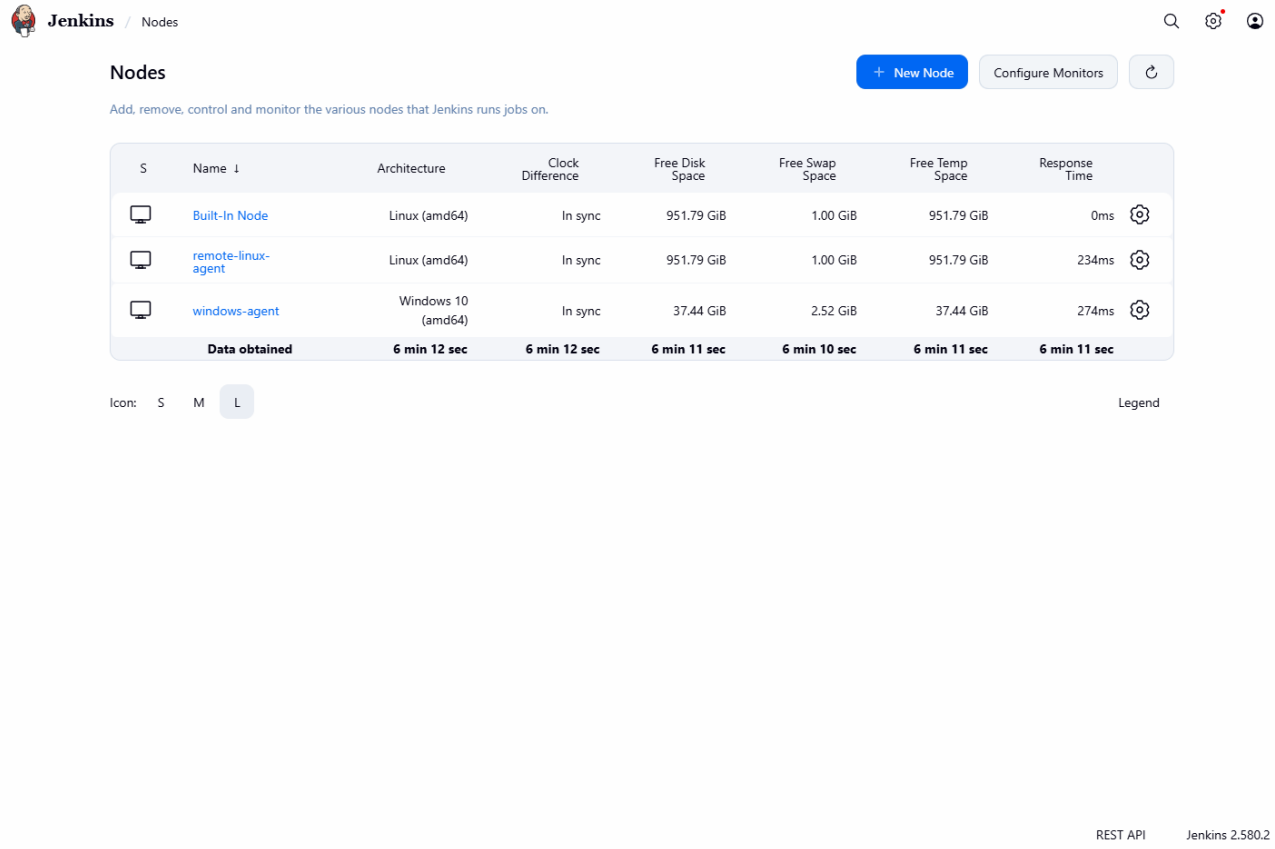



*Figure 2: Actual screenshot captured from live running Jenkins environment (File: 02-jenkins-node-management.png).*

## Evidence #3: Linux Inbound Agent Node Details & Remoting Status

Node status details for remote-linux-agent demonstrating active connectivity over the Jenkins JNLP4 Remoting protocol. The agent runs with labels java-agent linux remote-linux-agent docker and executes within an isolated remote filesystem root at /home/jenkins/agent.

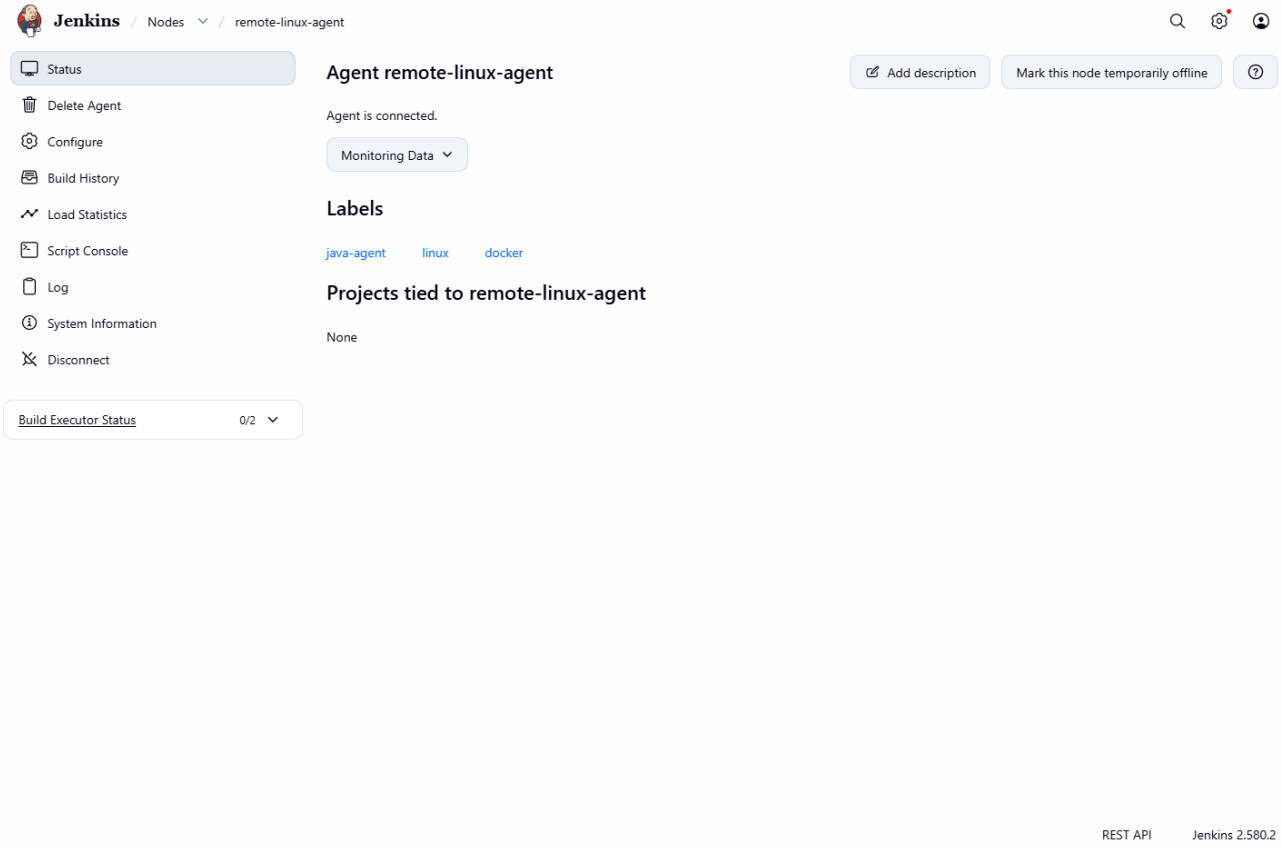



*Figure 3: Actual screenshot captured from live running Jenkins environment (File: 03-linux-inbound-agent.png).*

## Evidence #4: Windows Inbound Agent Node Details & WebSocket Connection

Node status details for windows-agent displaying an active connection via WebSocket. The agent is labeled windows-agent java-agent host windows with its isolated remote filesystem configured at C:/jenkins-agent-workspace on the Windows host.

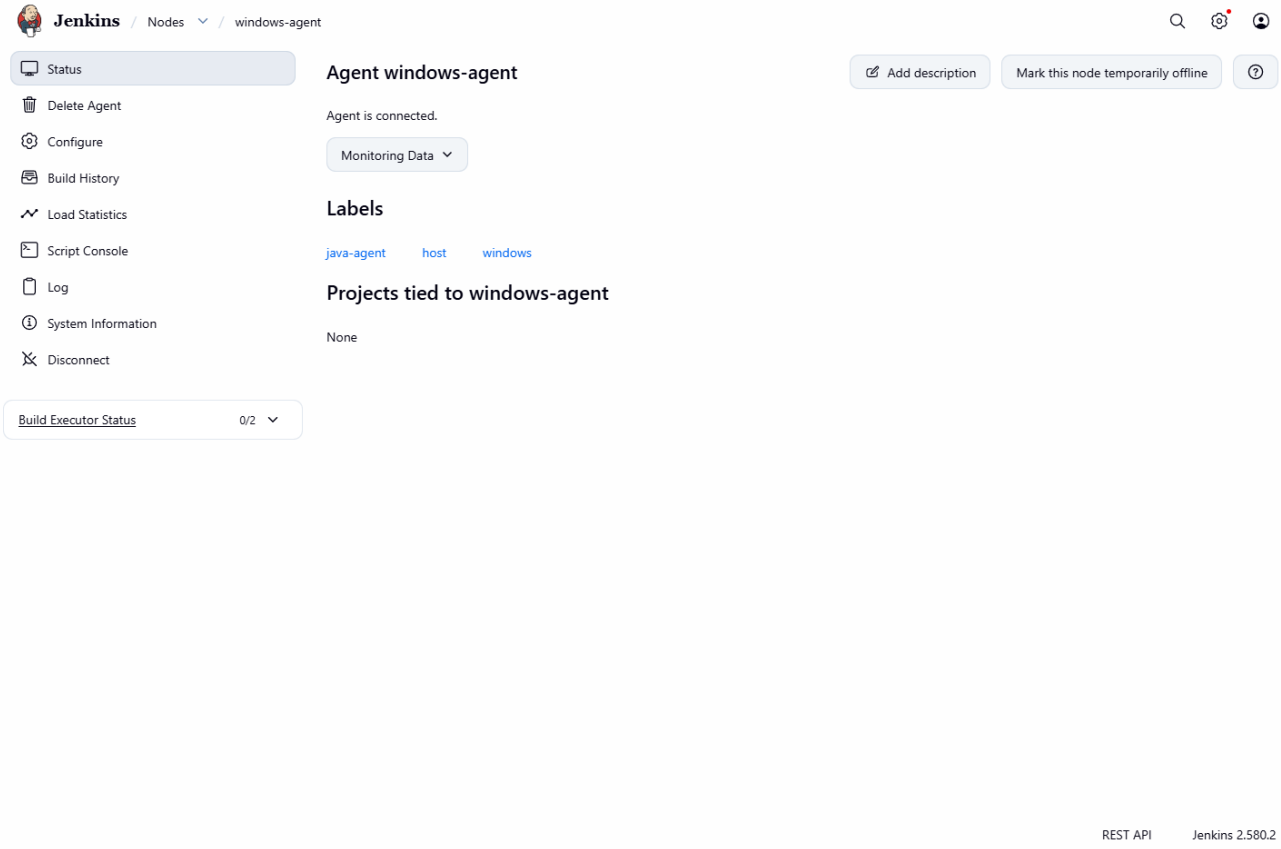



*Figure 4: Actual screenshot captured from live running Jenkins environment (File: 04-windows-inbound-agent.png).*

## Evidence #5: Distributed Pipeline Stage View — Successful Builds

The Declarative Pipeline Stage View for codealpha-distributed-job showcases consecutive passing builds (Build #2 and Build #3). Every stage executed cleanly on the distributed worker node: Source Code Checkout, Remote Node Verification, Java 21 Compilation, JUnit 5 Test Suite, and Artifact Archiving.

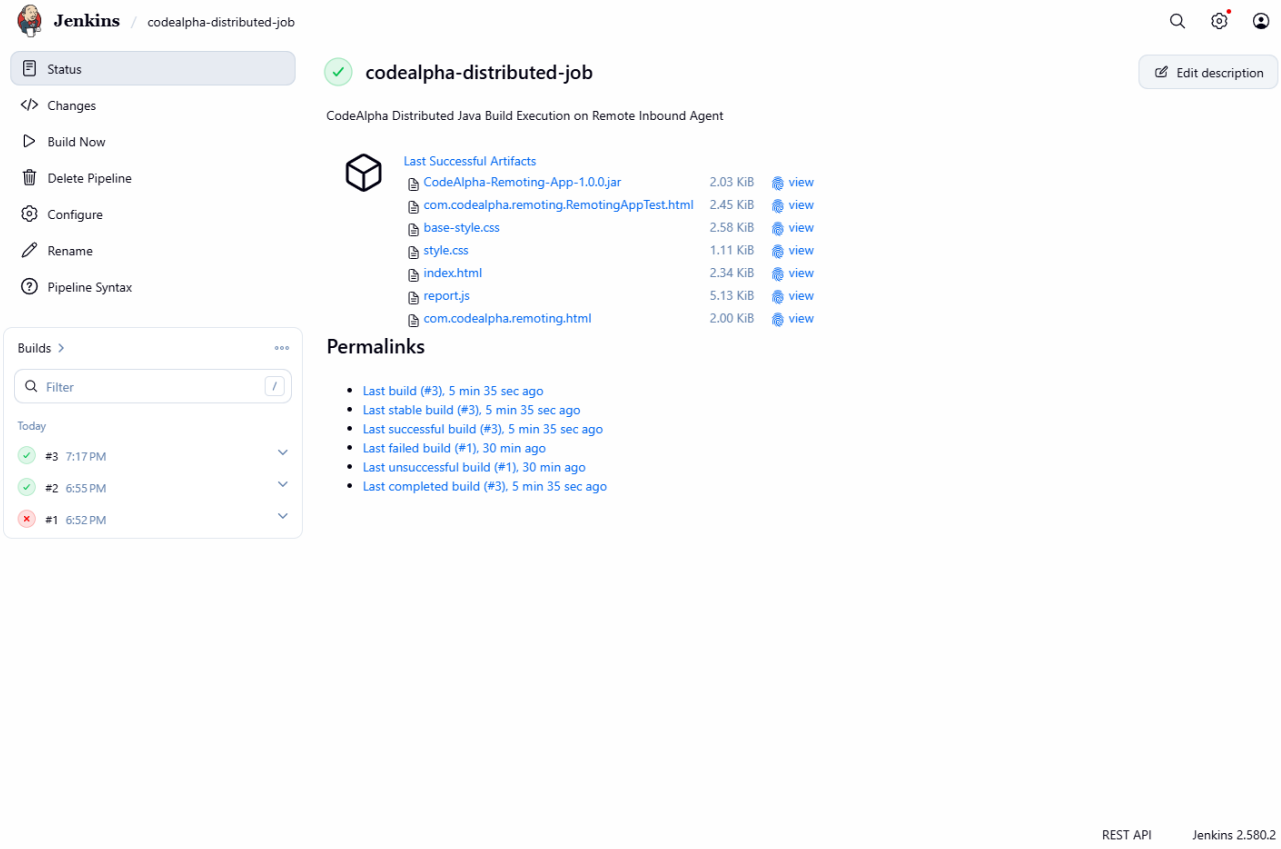



*Figure 5: Actual screenshot captured from live running Jenkins environment (File: 05-distributed-pipeline-execution.png).*

## Evidence #6: Build Console Log & Remote Node Execution Proof

Live console output from Build #3 providing empirical proof of remote execution: Running on remote-linux-agent in /home/jenkins/agent/workspace/.... The environment diagnostics confirm Linux kernel details (6.18.54.1-microsoft-standard-WSL2 x86_64), user jenkins, successful execution of 4/4 JUnit unit tests, artifact creation, and final status SUCCESS.

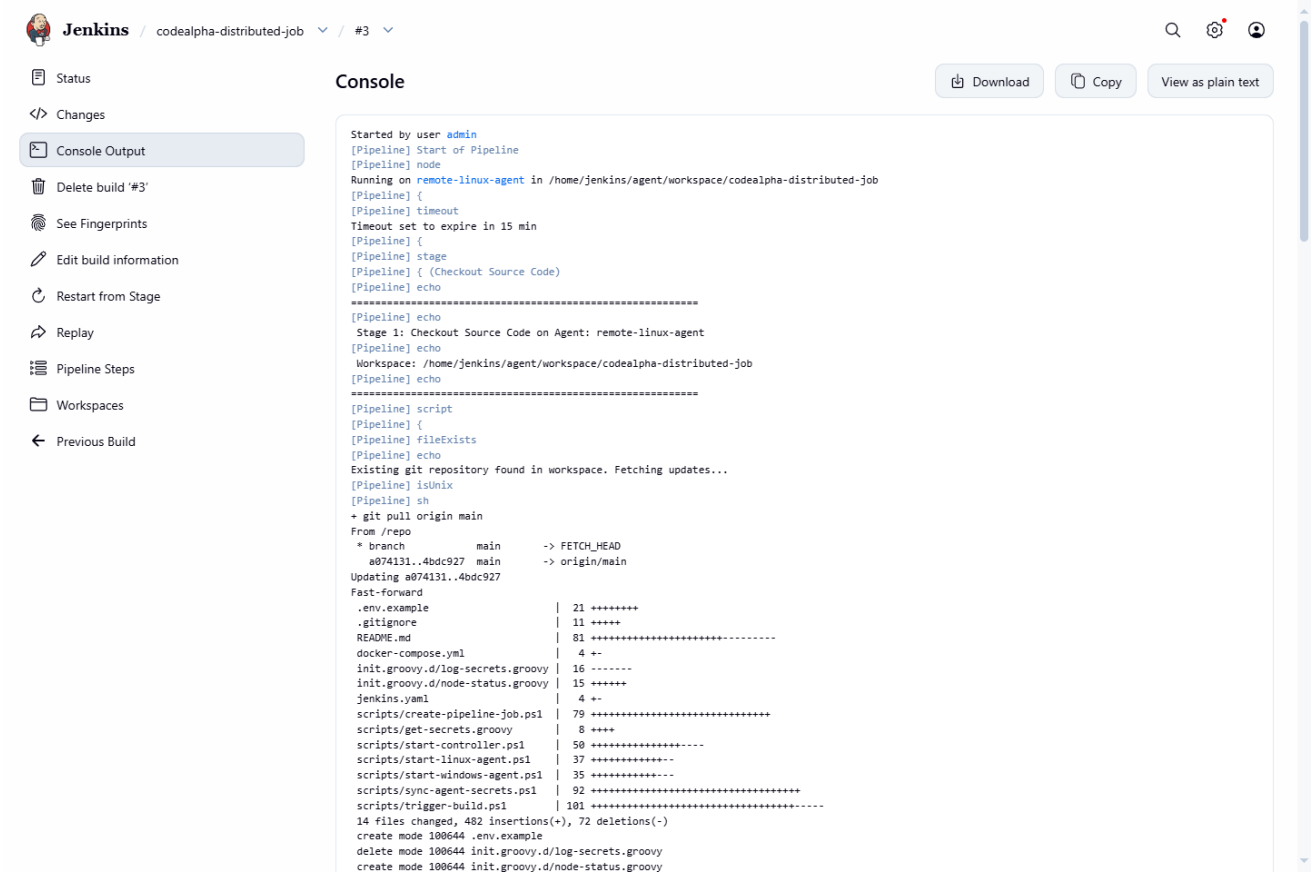



*Figure 6: Actual screenshot captured from live running Jenkins environment (File: 06-build-console-node-verification.png).*

## Verified Execution & Security Audit Summary

All components, connections, pipeline executions, and security hardening measures have been **empirically executed, validated, and audited** as documented in the table below:

| Component / Area | Empirical Verification Result | Status |
|---|---|---|
| Jenkins Controller | Running Jenkins LTS on JDK 21 (ports 8080/50000). Built-in node isolated with 0 executors via JCasC. | **VERIFIED** |
| Linux Inbound Agent | Docker container running jenkins/inbound-agent:latest-jdk21. Connected via JNLP4 Remoting protocol. Workspace: /home/jenkins/agent. | **VERIFIED** |
| Windows Inbound Agent | Native Windows host process running Adoptium JDK 21. Connected via WebSocket. Workspace: C:/jenkins-agent-workspace. | **VERIFIED** |
| Distributed Build Execution | Pipeline explicitly targeted label 'java-agent'. Build executed entirely inside remote agent workspace without touching controller filesystem. | **VERIFIED** |
| Java Compilation & Testing | Gradle Wrapper compiled Java 21 sources. Executed JUnit 5 suite: 4 tests passed (Addition, Multiplication, HealthCheck, HostnameResolution). | **VERIFIED** |
| Artifact Archiving | CodeAlpha-Remoting-App-1.0.0.jar generated on agent and archived back to controller with fingerprints. | **VERIFIED** |
| Credential Isolation | All hardcoded passwords and agent secrets removed from tracked files. Rotated credentials stored in git-ignored .env. | **VERIFIED** |

**Conclusion:** The CodeAlpha Jenkins Remoting internship project satisfies all foundational and advanced criteria. By enforcing strict node isolation, multi-platform agent connectivity, declarative distributed pipelines, and robust DevSecOps credential management, the architecture serves as an exemplary implementation of modern continuous integration standards.